# Redes de Datos — Preguntas y respuestas de Medín

2do parcial de teoría · Com. 403 · Mg. Cristian Medín · Transporte (y lo que mezcla de Red)

## Parte 1 — El parcial que tomó hace poco (oct 2026, otra comisión: las preguntas 1, 2, 3 y 5 ya las había tomado en 2024)

### 1. Enumere los propósitos de la capa de transporte y desarrolle al menos uno de ellos.

2024 y 2026

La capa de transporte (capa 4) es la **primera que trabaja extremo a extremo**: comunica directamente la aplicación del host origen con la del host destino, mientras que las capas 1 a 3 trabajan salto a salto. Es el enlace entre la capa de aplicación y las capas inferiores. Según el apunte de Medín, sus propósitos son:

1. **Seguimiento de conversaciones individuales:** un host tiene muchas aplicaciones comunicándose a la vez, y la capa 4 mantiene separado cada flujo de comunicación entre ellas.
2. **Segmentación de datos:** divide el flujo de datos de la aplicación en partes manejables, los **segmentos**, y le agrega a cada uno su encabezado de capa 4.
3. **Reensamble de segmentos:** en el receptor, reconstruye el flujo completo y en orden antes de entregárselo a la aplicación.
4. **Identificación de las aplicaciones:** mediante el **número de puerto**.

A esto los teóricos suman los servicios que lo hacen posible: **control de errores** y **control de flujo** extremo a extremo, **multiplexación** de varias conexiones sobre una misma IP, establecimiento y liberación de conexiones, y **control de congestión** (en TCP, junto con la capa 3).

**Desarrollo: identificación de las aplicaciones.** IP lleva el paquete hasta el host, pero no sabe a qué programa va. Para eso la capa 4 identifica cada aplicación con un **puerto**, un número de **16 bits** (0 a 65.535) que viaja en el encabezado de TCP y de UDP como puerto de origen y puerto de destino. La combinación **IP + puerto** se llama **socket** e identifica un extremo de la comunicación. La IANA divide los puertos en **conocidos** (0–1023, servicios estándar: HTTP 80, HTTPS 443, FTP 20/21, SMTP 25, DNS 53), **registrados** (1024–49.151) y **efímeros** (49.152–65.535, los que toma el cliente como origen). *Ejemplo:* al abrir una página, el segmento va a `IP del servidor : 443` y sale de `mi IP : 51.234`; la respuesta vuelve a ese puerto efímero, y así el sistema operativo sabe que es para el navegador y no para el mail o el Zoom, que usan la misma IP al mismo tiempo.

> En 2024 la misma pregunta salió como "¿Qué **responsabilidades** tiene la capa de transporte?". El apunte de Medín presenta los propósitos justamente como "las responsabilidades principales que debe cumplir", así que se contesta igual.

### 2. ¿Cómo controla la congestión el protocolo UDP?

2024 y 2026

**No la controla.** Es una pregunta trampa. UDP es un protocolo **sin conexión** y de "mejor esfuerzo" (*best effort*) que no implementa **control de congestión ni control de flujo**: no tiene ventana, ni ACK, ni números de secuencia, así que no tiene forma de enterarse de que la red está saturada. Envía los datagramas **al ritmo que le marca la aplicación**, sin adaptarse al estado de la red, y por eso puede contribuir a saturarla.

**Por qué es así:** UDP está pensado para aplicaciones donde importa más llegar a tiempo que llegar completo (voz, video en vivo, DNS). Bajar la velocidad o esperar confirmaciones agregaría retardo. Su encabezado de 8 bytes solo trae puertos, longitud y checksum: no hay ningún campo para regular el envío.

**Si hace falta control, lo hace la aplicación.** *Ejemplo:* en streaming, **RTCP** (el protocolo de control de RTP, que viaja sobre UDP) da realimentación sobre la calidad de la recepción, y con eso la aplicación **baja la velocidad y la calidad** del video cuando la red tiene problemas.

> Contraste con TCP, que sí controla la congestión: ventana de congestión, *slow start* (la ventana se duplica en cada RTT), incremento aditivo y, ante una pérdida, la ventana baja a la mitad (**AIMD**); variantes Tahoe y Reno. Mencionarlo suma fundamentación.

### 3. ¿Qué sucede si UDP detecta un error? ¿Y si lo detecta TCP?

2024 y 2026

Los dos detectan errores con el **checksum**: una suma en complemento a uno de palabras de 16 bits, calculada sobre el encabezado, los datos y un **pseudoencabezado IP** (IP origen, IP destino, número de protocolo y longitud). El receptor la recalcula y, si no coincide, el segmento llegó dañado. La diferencia está en qué hace cada uno después.

**UDP:** el checksum es **opcional** (si el emisor no lo calcula, el campo va en ceros y no hay control). Si detecta un error, **no lo corrige ni lo retransmite**, porque no tiene ACK, ni números de secuencia, ni temporizadores. Como dice la diapositiva de Medín: **"si detecta errores, no corrige pero avisa a las capas superiores"**, que son las que deciden qué hacer. En la práctica el datagrama dañado se **descarta**, y si la aplicación necesita ese dato, lo tiene que volver a pedir ella misma (por ejemplo, un cliente DNS que repite la consulta).

**TCP:** el checksum es **obligatorio**. El receptor **descarta** el segmento dañado y **no lo confirma**. El emisor guarda una copia de todo lo enviado hasta recibir su ACK; como el ACK no llega, **retransmite** el segmento cuando vence el **temporizador de retransmisión**, o antes si recibe **3 ACK duplicados**. Es el mecanismo **ARQ** (*Automatic Repeat reQuest*): ventana deslizante + números de secuencia + reenvío por timeout. Gracias a los números de secuencia, el receptor reordena y entrega a la aplicación un flujo de bytes **completo, sin duplicados y en orden**. La aplicación nunca se entera del error.

**En síntesis:** UDP detecta y deja la decisión a la capa superior; TCP detecta y recupera solo.

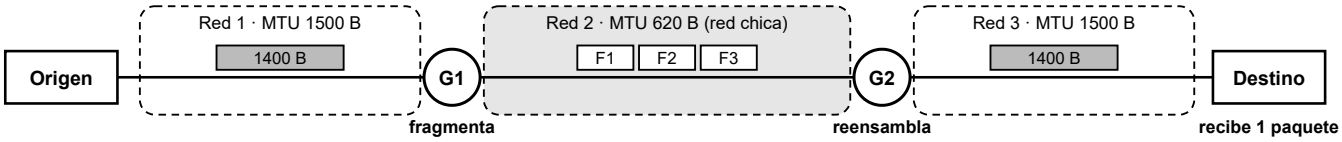
> El documento de preguntas de Medín de 2024 decía "UDP simplemente lo descarta". La diapositiva que mandó para este parcial dice "avisa a las capas superiores". La respuesta de arriba cubre las dos: no corrige, avisa y el dato dañado no se usa.

## 4. Explique al menos uno de los tipos de fragmentación que existen en una comunicación entre redes que tienen distintas tecnologías.

nueva · capa de red

**El problema.** Cada tecnología de red tiene un tamaño máximo de paquete, la **MTU** (*Maximum Transmission Unit*): por ejemplo, 1500 bytes en Ethernet. Cuando un paquete grande pasa de una red con MTU grande a otra con MTU más chica, el router que las une tiene que **dividirlo en fragmentos**, cada uno enviado como un paquete aparte. Hay **dos tipos** de fragmentación, que se distinguen por **dónde se reensamblan los fragmentos**:

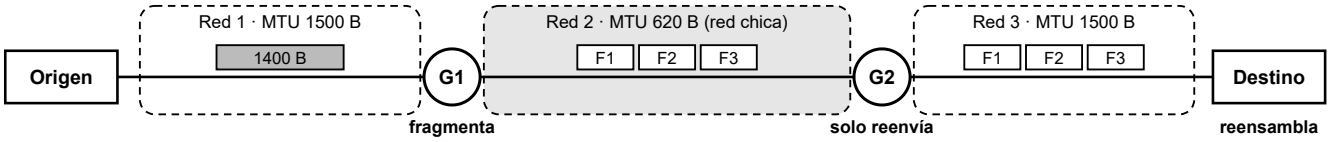
**(a) TRANSPARENTE: el router de salida de la red chica reensambla**




*Todos los fragmentos deben salir por G2. La Red 3 ni se entera de que hubo fragmentación.*

---

**(b) NO TRANSPARENTE (la de IP): solo el host destino reensambla**




*Cada fragmento es un datagrama independiente (puede ir por otra ruta). Se rearma con Identificación, offset y MF.*

**1) Fragmentación transparente.** El router de entrada a la red de paquetes chicos (G1) fragmenta, y **todos los fragmentos tienen que salir por el mismo router de salida** (G2), que los **vuelve a unir** antes de mandarlos a la red siguiente. El paso por la red chica es "transparente": las redes posteriores ni se enteran de que hubo fragmentación. *Desventajas:* el router de salida necesita saber cuándo llegaron todas las piezas (un contador o un bit de "fin de paquete"); las **rutas quedan restringidas**, porque todos los fragmentos deben ir al mismo router; los routers hacen **mucho trabajo** y gastan buffer; y si el paquete cruza varias redes chicas, se fragmenta y reensambla una y otra vez.

**2) Fragmentación no transparente.** Es la que usa **IP**. Una vez fragmentado, **cada fragmento viaja como un datagrama independiente**, incluso por rutas distintas, y **solo el host destino lo reensambla**. Para poder rearmarlo, el encabezado IPv4 trae:

- **Identificación:** el mismo número en todos los fragmentos del mismo datagrama.
- **Desplazamiento** (*fragment offset*, 13 bits): la posición del fragmento dentro de los datos originales, en unidades de 8 bytes.
- **MF** (*More Fragments*): vale 1 en todos los fragmentos menos en el último. **DF** (*Don't Fragment*): si vale 1, el router no fragmenta, descarta el paquete y avisa por ICMP.

*Ventajas:* los **routers trabajan menos**; los fragmentos pueden llegar **desordenados** (el destino los acomoda por el desplazamiento); un fragmento se puede **volver a fragmentar** si encuentra una red con MTU todavía menor. *Desventajas:* más sobrecarga, porque cada fragmento lleva su propio encabezado hasta el final; y si se pierde **un**

**solo fragmento**, se pierde el datagrama entero: el destino arranca un **temporizador de reensamblado** y, si vence, descarta lo recibido.

*Ejemplo:* un datagrama de 1400 B de datos (+20 B de encabezado) pasa a una red con MTU 620 B. Entran 600 B de datos por fragmento (múltiplo de 8):

| Fragmento | Longitud total | Bytes de datos | Desplazamiento (campo = bytes ÷ 8) | MF |
|---|---|---|---|---|
| 1 | 620 | 0 – 599 | 0 | 1 |
| 2 | 620 | 600 – 1199 | 75 | 1 |
| 3 | 220 | 1200 – 1399 | 150 | 0 |

**Para sumar:** hoy Internet trata de **evitar** la fragmentación con el **descubrimiento de la MTU de la ruta**. La fuente manda los paquetes con DF = 1; si un router no puede pasarlos, los descarta y devuelve un error ICMP con la MTU que acepta, y la fuente achica sus paquetes. Por eso TCP usa segmentos de hasta **1460 B** (1500 − 20 de IP − 20 de TCP): entran en una trama Ethernet sin fragmentar.
Fuente: Tanenbaum, 5ª ed., §5.5.5. El apunte de Medín solo trae los campos DF, MF y desplazamiento; la clasificación en transparente y no transparente es la del libro.

### 5. Mencione las principales diferencias entre TCP y UDP.

2024 y 2026

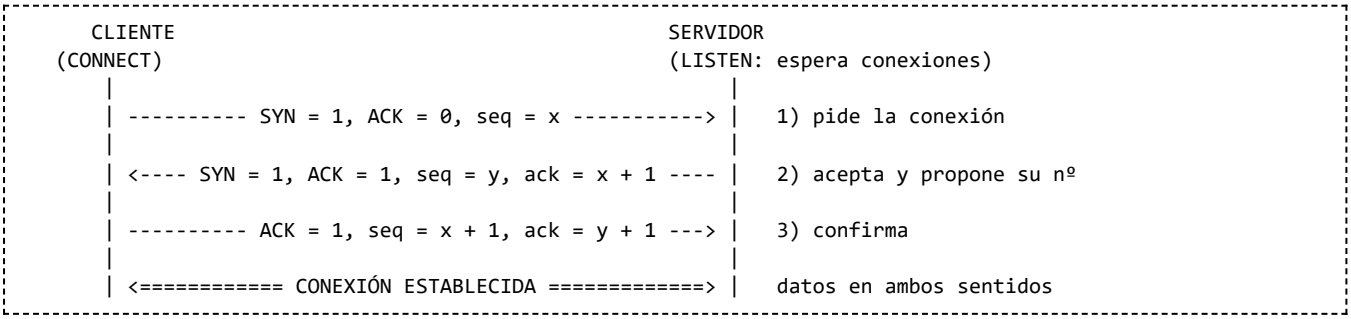
| Aspecto | TCP | UDP |
|---|---|---|
| **Conexión** | Orientado a conexión: establece con el **3-way handshake** y libera con FIN/ACK en cada sentido | Sin conexión: envía directamente |
| **Confiabilidad** | Confiable: confirma con ACK, retransmite lo perdido, ordena y descarta duplicados | No garantiza entrega, orden ni ausencia de duplicados (mejor esfuerzo) |
| **Control de errores** | Checksum **obligatorio** + retransmisión (ARQ) | Checksum **opcional**; no corrige, avisa a la capa superior |
| **Control de flujo** | Sí, con ventana deslizante (campo *Window Size*) | No |
| **Control de congestión** | Sí: slow start, AIMD, Tahoe/Reno | No; si hace falta, lo hace la aplicación |
| **Encabezado** | **20 bytes** fijos + hasta 40 de opciones (secuencia, ACK, flags, ventana…) | **8 bytes**: puerto origen, puerto destino, longitud y checksum |
| **Tipo de servicio** | Flujo de bytes (*byte stream*) | Mensajes independientes (datagramas) |
| **Destinatarios** | Solo unicast, full-duplex, punto a punto | Unicast, broadcast y multicast |
| **Velocidad / costo** | Más lento: handshake, ACK y control de congestión | Más rápido y liviano |
| **Usos** | Web (HTTP 80, HTTPS 443), correo (SMTP 25), FTP (20/21), SSH, transferencias bancarias | Voz y video en tiempo real (RTP), DNS (53), DHCP (67/68), juegos, VPN, QUIC/HTTP3 |

**Para cerrar la respuesta:** TCP se usa cuando no puede faltar ni desordenarse nada (un archivo o una transferencia bancaria); UDP, cuando importa más que llegue a tiempo que completo (en una videollamada, retransmitir un cuadro viejo no sirve).

## Parte 2 — La otra pregunta del 2do parcial de 2024 (29/10/2024, com. 403; las otras cuatro son las 1, 2, 3 y 5 de arriba)

### 6. Grafique un ejemplo de conexión 3-way handshake.

2024



1. El servidor está en **LISTEN**. El cliente ejecuta **CONNECT** y envía un segmento con **SYN = 1** y su número de secuencia inicial **x** (además informa el puerto destino y el MSS).
2. El servidor responde con **SYN = 1 y ACK = 1** (SYN-ACK): confirma el número del cliente con **ack = x + 1** y propone su propio número de secuencia inicial **y**.
3. El cliente responde con **ACK = 1, ack = y + 1**. Desde ahí la conexión es **full-duplex**.

**Por qué tres pasos:** cada lado tiene que elegir su número de secuencia y que el otro se lo confirme. Así se evita que un segmento **duplicado y demorado** de una conexión vieja abra una conexión falsa (por ejemplo, repetir una transferencia bancaria). Los números de secuencia son de **32 bits** y el inicial es **aleatorio**, para que nadie pueda predecirlo.

> **Si pide también la desconexión:** es **simétrica**: cada sentido se cierra por separado con **FIN** + **ACK** (4 segmentos). Por el **problema de los dos ejércitos** nunca se puede asegurar que el último mensaje llegó, así que cada lado termina de cerrar por **timeout**. **Ataque relacionado:** la **inundación SYN** manda muchos SYN y nunca completa el paso 3; el servidor se llena de conexiones a medio abrir (DoS). Se mitiga con **SYN cookies** y limitando las conexiones nuevas.